Project Summary and Reflections Report

Michael Heiser – 10/14/2026

Summary:

To what extent was your approach aligned to the software requirements?

My testing approach aligned with the software requirements by breaking down requirements into more manageable and clear parts, then interpreting them in the context of the system as a whole. For instance, the task object had to contain an ID that was unique, no larger than 10 characters, non-updatable, and not null. This requirement could be broken down into two parts. The first part being object requirements that can be tested on the class object itself (object ID field cannot be null or more than 10 characters and cannot be updated). The second part is requirements that must be tested using an outside class or system (each object ID must be unique). By further breaking down requirements into manageable parts I can more accurately align my testing approach to the software requirements.

How do you know your JUnit tests were effective based on the coverage percentage?

At Google, 60% and higher is generally considered an acceptable coverage percentage for many products (Google, 2020). The coverage percentage for each of my JUnit tests was 100%. So, if we are basing test effectiveness solely on coverage, my JUnit tests are incredibly effective.

How did you ensure that your code was technically sound?

To ensure my code was technically sound I used JUnit tests. The JUnit tests I wrote ensured each service was error free and that each decision path was reached and produced its expected output.

How did you ensure that your code was efficient?

For this project I tried to aim for readability more than efficiency. However, there were some approaches I took to ensure efficiency and minimal resource usage. For example, in each service class I used a hash map to store their associated objects. The hash map provided quick look up times and effective implementation of the unique ID requirement.

Reflection:

What were the software testing techniques that you employed in this project?

The first technique I used was total coverage testing, which is when you check that every decision path was reached. Using JUnit test coverage, I was able to ensure that every branch of code was tested and produced the expected output. After getting accustomed to how JUnit testing works, I was able to employ experience-based error guessing in later milestones. I could now anticipate errors and mistakes that might be difficult to pinpoint later on thanks to learning and adapting from prior testing issues.

What are the other software testing techniques that you did not use for this project?

I hardly used any black-box testing techniques in this project apart from error guessing. In my eyes, the project scope was too small, and the requirements were too simple to really benefit from formal planning techniques and in-depth analyses of how the system will take and process inputs.

For each of the techniques you discussed, explain the practical uses and implications for different software development projects and situations.

Coverage testing is widely used throughout the industry to measure the amount of code both tested and untested and provides measurable results regarding the software's quality (GeeksforGeeks, 2026a). While it does have its drawbacks, it is still a powerful tool in ensuring the quality of all kinds of projects in all kinds of situations. Experience-based error guessing provides an informal yet easy to apply technique that can prove invaluable when trying to prevent critical errors (GeeksforGeeks, 2026b). However, this technique is based on the individual tester's skills and can provide widely varying results.

In acting as a software tester, to what extent did you employ caution? Why was it important to appreciate the complexity and interrelationships of the code you were testing?

When acting as a software tester, I employed caution by closely following and reflecting on the system requirements. I made sure each test case was closely related to a requirement to some degree and through JUnit tests I was able to carefully ensure each decision path was reached a produced the expected result.

Assess the ways you tried to limit bias in your review of the code. On the software developer side, can you imagine that bias would be a concern if you were responsible for testing your own code?

Truthfully as both the developer and tester, I did exhibit some bias when writing the test cases. An example being how I knew the constructor for the objects used the setter functions when constructing an object, so by testing only the constructor for invalid inputs I knew I did not need to test the setter functions separately. It is common that software developers when testing their code exhibit some form of bias, whether intentional or not. This is because the software developer knows exactly how the code should work and likely holds to a mindset of "how can I make this code meet requirements". This is different from how professional testers should test

code. Testers should adopt a mindset of "how can I break this code", while either not knowing or pretending not to know how the code works internally.

Evaluate the importance of being disciplined in your commitment to quality as a software engineering professional. Why is it important not to cut corners when it comes to writing or testing code? How do you plan to avoid technical debt as a practitioner in the field?

In the long term, cutting corners when writing and testing code will almost always come back to bite you. When you leave code unrefined and untested and continue to build features and components, the risk and impact of that neglected code drastically increases. Later down the line issues will likely arise, and because of all the added features and dependencies that neglected code will be significantly more complex and costly to fix than if it was just properly written and tested from the start. To reduce or avoid technical debt, many articles like "*Avoiding Technical Debt with These 'Core Four' Practices"* recommend that software development teams openly communicate existing intents, write code designed to be changeable, and hold to software development standards like Test-Driven Development (2021).

References:

Google. (2020, August 7). *Code coverage best practices*. Google Testing Blog. https://testing.googleblog.com/2020/08/code-coverage-best-practices.html

GeeksforGeeks. (2026a, June 15). *Code coverage testing in software testing*. GeeksforGeeks. https://www.geeksforgeeks.org/software-engineering/code-coverage-testing-in-software-testing/

GeeksforGeeks. (2026b, June 9). *Error guessing in software testing*. GeeksforGeeks. https://www.geeksforgeeks.org/software-testing/error-guessing-in-software-testing/

Avoiding Technical Debt with These "Core Four" Practices. (2021, April 26). Scrum Alliance. https://resources.scrumalliance.org/Article/avoiding-technical-debt-core-four-practices